

Parallel Algorithm Design

Below is a **structured, detailed, and exam-oriented explanation** of **Chapter 3: Principles of Parallel Algorithm Design**, rewritten for clarity while preserving the technical depth and intent of the original text.

No content is skipped; ideas are expanded where conceptual understanding is critical.

Chapter 3

Principles of Parallel Algorithm Design

Parallel algorithm design focuses on **solving problems efficiently using multiple processors simultaneously**. Unlike sequential algorithms, which specify a single ordered sequence of steps, parallel algorithms must explicitly define **which steps can execute concurrently, how tasks interact, and how work is distributed across processing units**.

Designing an effective parallel algorithm is not simply about splitting work—it requires careful consideration of:

- concurrency,
 - communication,
 - synchronization,
 - and mapping to hardware.
-

1. Sequential vs Parallel Algorithms

Sequential Algorithm

- A **linear recipe** of steps.
- Executes on **one processor**.
- No overlap in execution.

Parallel Algorithm

- A **concurrent recipe**.
- Executes on **multiple processors**.
- Must explicitly specify:
 - which computations can run simultaneously,
 - how data is shared or distributed,
 - when processors must synchronize.

Performance improvement is achieved **only if concurrency is exposed and effectively utilized**.

2. Key Challenges in Parallel Algorithm Design

Designing a parallel algorithm typically involves the following decisions:

1. **Identifying concurrent work**
2. **Decomposing computation into tasks**
3. **Mapping tasks to processes**
4. **Managing shared or distributed data**
5. **Synchronizing execution**
6. **Minimizing communication overhead**

Although many choices exist at each step, **only a few combinations yield good performance**, and optimal choices often depend on:

- architecture (shared memory vs distributed memory),
- programming paradigm,
- problem size.

Automatic parallelization tools are currently effective only for highly structured code; hence, **manual design remains essential**.

3.1 Preliminaries

Parallel algorithm design fundamentally consists of **two steps**:

1. **Decomposition** – breaking computation into tasks
 2. **Mapping** – assigning tasks to processes
-

3.1.1 Decomposition, Tasks, and Dependency Graphs

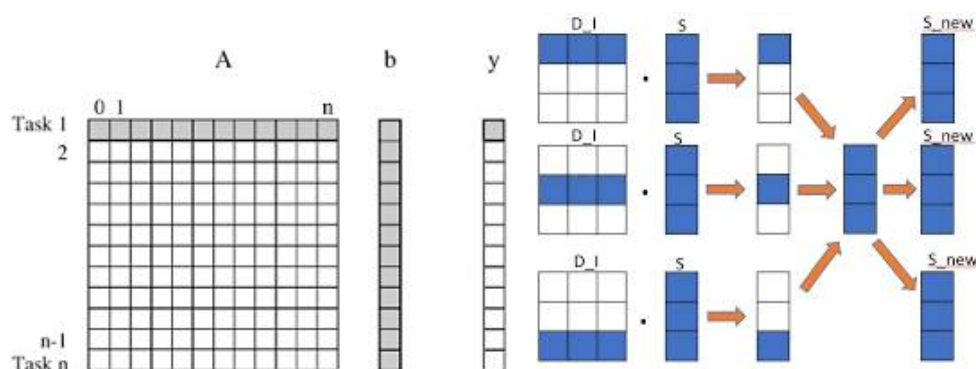
Decomposition

Decomposition is the process of **dividing a computation into smaller parts**, some or all of which can run in parallel.

Tasks

- A **task** is a programmer-defined unit of computation.
 - Tasks are **indivisible once defined**.
 - Tasks may vary in size.
-

Example: Dense Matrix-Vector Multiplication



For computing

$$y = A \cdot b$$

- Each output element $y[i]$ is the dot product of row i of matrix A with vector b .
- Each $y[i]$ can be treated as an **independent task**.
- Alternatively, rows can be grouped to form **coarser tasks**.

This example shows that **multiple decompositions are possible** for the same problem.

Task-Dependency Graph (TDG)

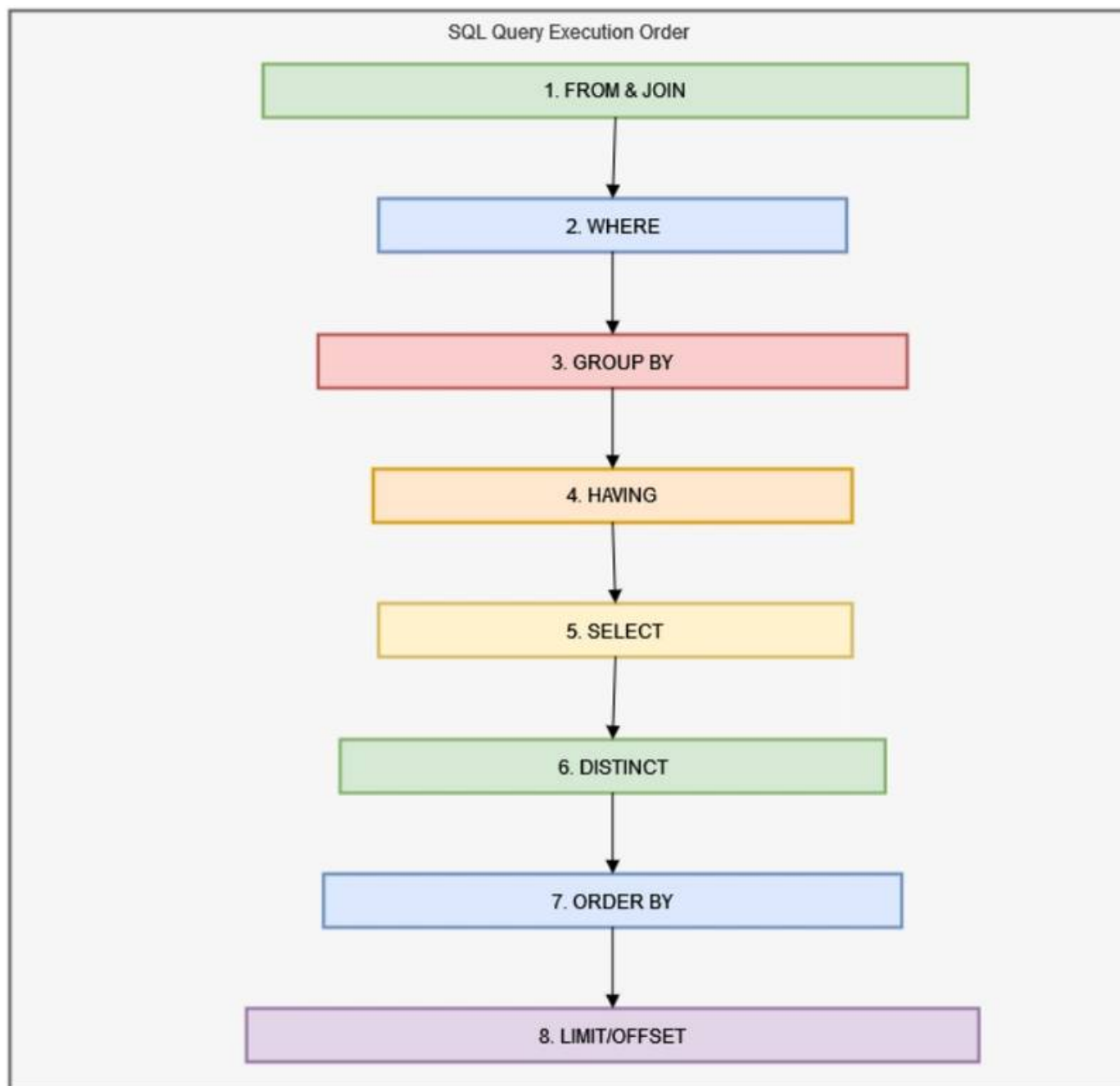
A **task-dependency graph** is a **directed acyclic graph (DAG)** where:

- nodes represent tasks,
- edges represent dependencies,
- a task can execute only after all incoming edges are resolved.

Key points:

- TDGs may be disconnected.
- TDGs may have no edges (fully independent tasks).
- TDGs define **execution order constraints**, not processor assignment.

Example: Database Query Processing



Processing a query such as:

```
java
```

```
MODEL = "Civic" AND
YEAR = "2001" AND
(COLOR = "Green" OR COLOR = "White")
```

Steps:

1. Create intermediate tables for each condition.
2. Perform intersections and unions.
3. Final intersection yields result.

Each intermediate table corresponds to a **task**, and dependencies arise when one task needs output from another.

Different **equivalent logical expressions** produce **different task-dependency graphs**, affecting concurrency and performance.

3.1.2 Granularity, Concurrency, and Task Interaction

Granularity

Granularity refers to **task size**:

- **Fine-grained decomposition**
 - Many small tasks
 - High potential concurrency
 - Higher communication overhead
 - **Coarse-grained decomposition**
 - Few large tasks
 - Lower concurrency
 - Less communication overhead
-

Degree of Concurrency

- **Maximum degree of concurrency**
→ Maximum number of tasks executable at the same time.
- **Average degree of concurrency**
→ Average parallelism over entire execution.

Average concurrency is a **better predictor of performance**.

Critical Path

The **critical path** is:

- the **longest weighted path** from a start node to a finish node in the task graph.

Definitions:

- **Critical path length** = sum of task weights along longest path
- **Average concurrency** =

$$\frac{\text{Total work}}{\text{Critical path length}}$$

Shorter critical paths → higher achievable parallelism.

Practical Limits to Parallelism

Parallel speedup is limited by:

1. Bounded granularity

- Finite number of operations

2. Task interactions

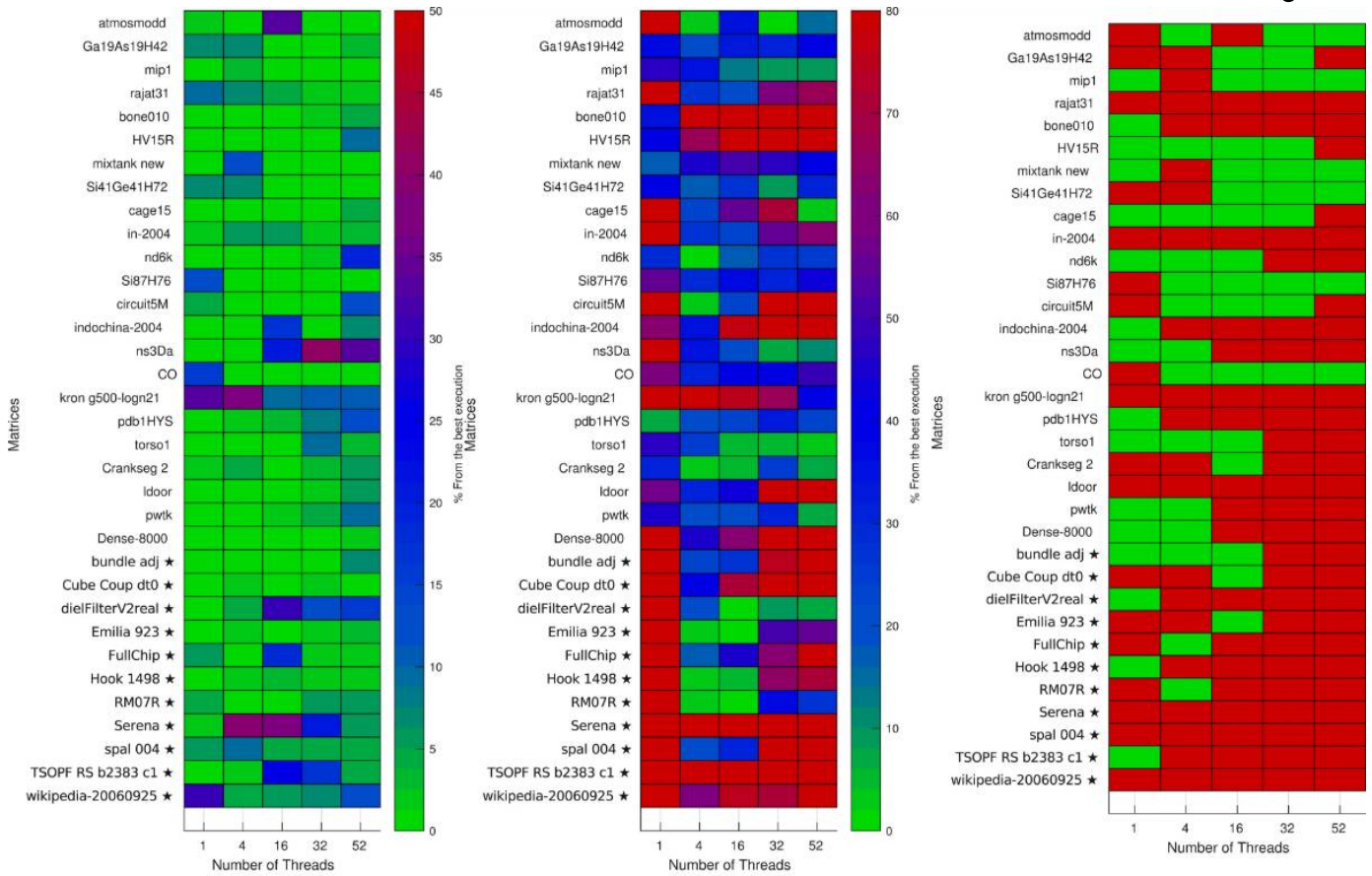
- Communication
- Synchronization
- Data sharing

Task-Interaction Graph

- Nodes = tasks
- Edges = data or control interactions
- Typically undirected
- Superset of dependency graph

Even independent tasks may interact due to **shared data access**.

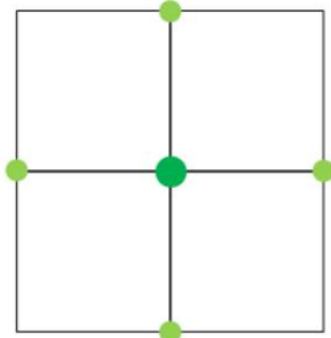
Example: Sparse Matrix-Vector Multiplication



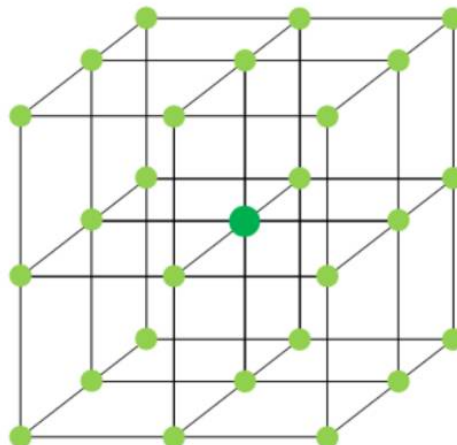
(A)

(B)

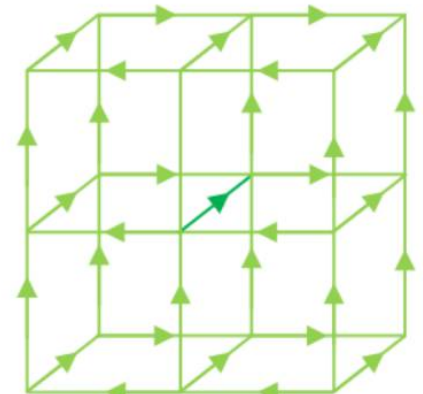
(C)



(a)



(b)



(c)

Each task:

- Computes one output element
- Owns one row of the matrix and one element of vector b
- Requires multiple $b[j]$ values from other tasks

Result:

- Heavy inter-task communication
- Nontrivial task-interaction graph

3.1.3 Processes and Mapping

Processes

- Logical computing agents
- Execute tasks
- May compute, communicate, and synchronize

Not equivalent to OS processes.

Mapping

Mapping assigns **tasks to processes**.

A good mapping aims to:

1. Maximize utilization of concurrency
2. Prioritize critical-path tasks
3. Minimize inter-process communication

These goals often **conflict**, making mapping a key optimization challenge.

3.1.4 Processes vs Processors

- **Processes:** abstract execution entities
- **Processors:** physical hardware units

Reasons for separation:

- Multi-core nodes
- Hybrid architectures
- Multi-level parallelism

Parallel algorithms are often designed **hierarchically**:

- Inter-node parallelism (message passing)
 - Intra-node parallelism (shared memory)
-

3.2 Decomposition Techniques

Parallelism is extracted using **decomposition techniques**. Common categories:

1. Recursive decomposition
2. Data decomposition

3. Exploratory decomposition
 4. Speculative decomposition
 5. Hybrid decomposition
-

3.2.1 Recursive Decomposition

Used for **divide-and-conquer algorithms**.

Characteristics:

- Problem split into independent subproblems
 - Subproblems solved recursively
 - Natural tree-shaped task graph
-

Example: Parallel Quicksort

- Partition step creates subarrays
- Recursive calls execute concurrently
- Concurrency increases deeper in recursion tree

Limitation:

- Top-level partition is sequential
 - Load imbalance possible
-

Example: Finding Minimum via Recursive Decomposition

Serial version:

- Linear scan
- No concurrency

Recursive version:

- Divide array
- Compute minimum of halves in parallel
- Combine results

This restructuring **creates concurrency where none existed before**.

3.2.2 Data Decomposition

Concurrency is derived by **partitioning data**, then mapping computations to partitions.

Types of Data Partitioning

1. Partitioning Output Data

- Each task computes a portion of output
- Common in matrix operations

2. Partitioning Input Data

- Each task processes a subset of input
- Results combined later

3. Partitioning Both Input and Output

- Higher concurrency
- Requires reduction steps

4. Partitioning Intermediate Data

- Introduces extra storage
 - Can increase concurrency
-

Owner-Computes Rule

Each task:

- Owns a data partition
- Performs all computations involving that data

Widely used in parallel systems.

3.2.3 Exploratory Decomposition

Used for **search problems**.

Key features:

- Search space partitioned
 - Tasks explore different regions
 - Execution stops when solution found
-

Example: 15-Puzzle

- State space search
- Initial expansion done serially

- Leaf states explored in parallel

Important property:

- Parallel work may be **less or more** than serial work
 - Leads to **anomalous speedups**
-

3.2.4 Speculative Decomposition

Used when:

- Future computation depends on unknown condition
- Likely branches are computed in advance

Characteristics:

- Some work may be discarded
 - Reduces latency of dependent stages
 - Common in:
 - discrete event simulation
 - pipelined systems
-

3.2.5 Hybrid Decompositions

Combines multiple techniques to improve performance.

Examples:

- Input decomposition + recursive reduction (minimum finding)
- Recursive + data decomposition (parallel quicksort)

Hybrid approaches are often **necessary for scalability**.

Summary

Parallel algorithm design requires balancing:

- concurrency,
- communication,
- synchronization,
- memory usage,
- and hardware characteristics.

No single decomposition strategy fits all problems. Effective designs:

- analyze task dependencies,
- minimize critical paths,
- reduce interaction overhead,
- and adapt to architecture.

This chapter provides the **conceptual foundation** required to reason about and design efficient parallel algorithms in practice.

Below is a **thorough, structured, and exam-ready explanation** of Sections 3.3 and 3.4 of *Principles of Parallel Algorithm Design*.

The explanation is **expanded conceptually, clearly formatted**, and **includes code snippets wherever they appear in the description**, exactly as requested.

3.3 Characteristics of Tasks and Interactions

After decomposing a problem into parallel tasks, the **next critical step** is deciding **how to map these tasks onto processes**.

A good mapping depends heavily on:

- the **nature of the tasks**, and
- the **way tasks interact with one another**.

Understanding these characteristics is essential to designing **efficient, scalable parallel algorithms**.

3.3.1 Characteristics of Tasks

Four task-related characteristics strongly influence mapping decisions.

1. Task Generation

Static Task Generation

- All tasks are known **before execution begins**
- Task-dependency graph is fixed
- Common in **data decomposition**

Examples

- Matrix multiplication
- LU factorization
- Recursive minimum finding (fixed input size)

Advantages

- Easier mapping
 - Lower runtime overhead
 - Enables static scheduling
-

Dynamic Task Generation

- Tasks are created **during execution**
- Task-dependency graph is not fully known in advance
- Task sizes and counts depend on input data

Examples

- Quicksort (task tree depends on pivot choices)
- Search problems (15-puzzle)
- Dynamic exploratory decomposition

Implication

- Requires **dynamic mapping**
 - Harder to load-balance statically
-

Static vs Dynamic in Exploratory Decomposition

Static approach

- Pre-generate search space to a fixed depth
- Assign resulting states as tasks

Dynamic approach

- Tasks expand states and generate new tasks recursively
 - Termination can occur early if solution is found
-

2. Task Sizes

Uniform Tasks

- All tasks take roughly equal time
- Easier to load-balance

Examples

- Matrix multiplication blocks

- Image processing kernels

Non-Uniform Tasks

- Tasks vary significantly in execution time
- Static mapping often causes imbalance

Examples

- Quicksort partitions
- Search-based tasks
- Sparse computations

3. Knowledge of Task Sizes

Task Size Knowledge	Mapping Strategy
Known beforehand	Static mapping possible
Unknown beforehand	Dynamic mapping preferred

Examples

- Matrix multiplication → known sizes
- 15-puzzle search → unknown depth and effort

4. Size of Data Associated with Tasks

Mapping must consider **where data lives** and **how large it is**.

Problem	Input Size	Output Size
15-Puzzle	Small	Small
Minimum finding	Large	Very small
Quicksort	Large	Large

Key Insight

Tasks with large data should avoid frequent reassignment (especially in message passing).

3.3.2 Characteristics of Inter-Task Interactions

Parallel tasks almost always interact.

The **type of interaction** strongly affects mapping and programming paradigms.

1. Static vs Dynamic Interactions

Static Interaction

- Who communicates with whom is known beforehand
- Timing is predictable

Examples

- Matrix multiplication
- LU factorization

Programming

- Easy in message passing
 - Simple synchronization
-

Dynamic Interaction

- Interaction partners or timing unknown beforehand
- Requires runtime coordination

Example

- 15-puzzle work stealing
- Dynamic task redistribution

Implication

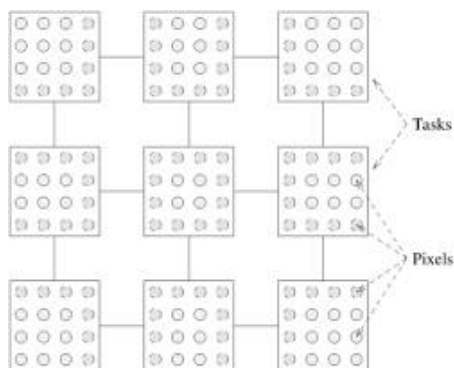
- Harder in message passing
 - Requires polling or extra synchronization
 - Easier in shared memory
-

2. Regular vs Irregular Interactions

Regular Interaction

- Structured, predictable patterns
- Exploitable for optimization

Example: Image Dithering



 <https://www.researchgate.net/profile/Mohammad-Al-Towaiq/publication/263797680/figure/fig3/AS%3A667835705602048%401536235848030/A-2D-mesh-NoC-architecture-k-4.png>

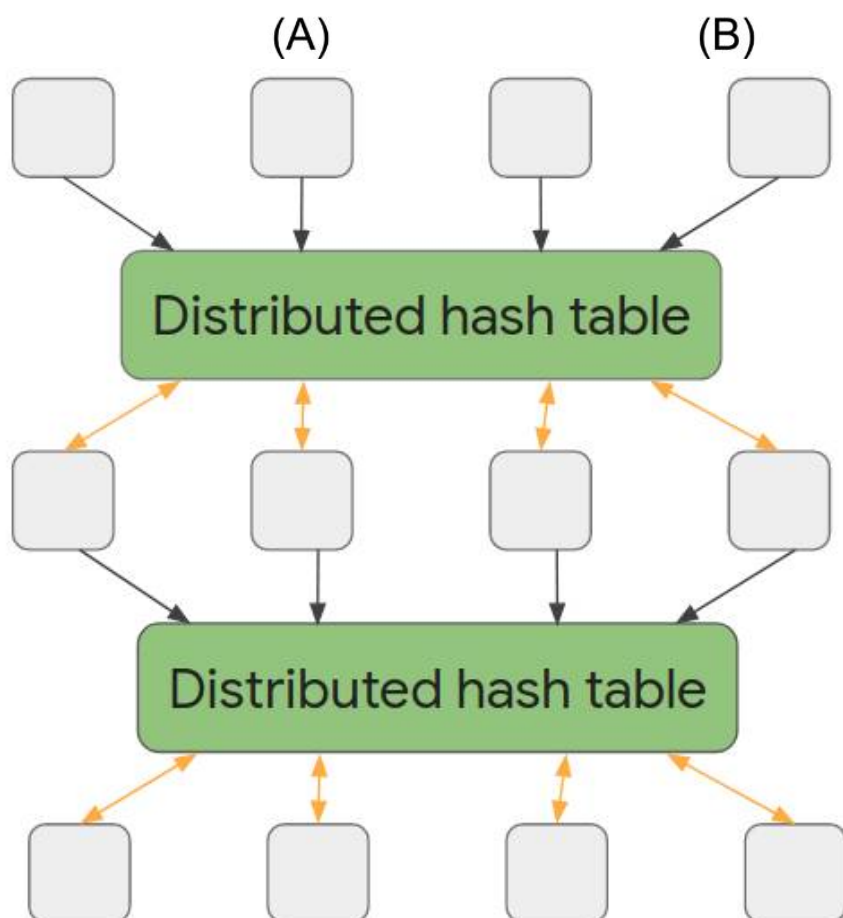
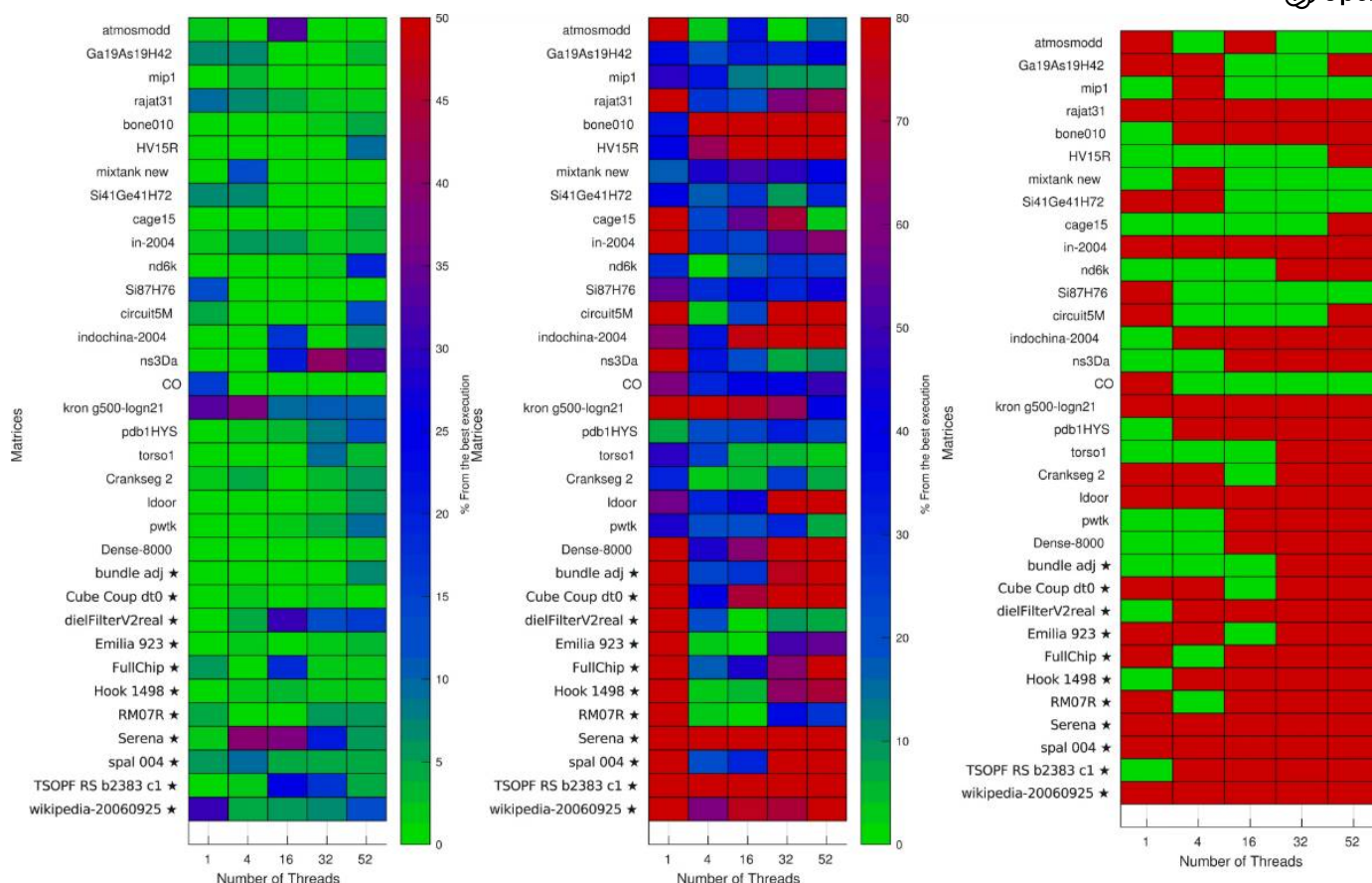
- Each task processes a block of pixels
- Needs boundary pixels from neighbors
- Interaction graph forms a **2D mesh**

Irregular Interaction

- Data-dependent, unpredictable access

Example

- Sparse matrix-vector multiplication
- Access pattern depends on non-zero structure



(C)

3. Read-Only vs Read-Write Interactions

Read-Only

- Shared data is never modified
- Safe and scalable

Examples

- Input matrices in matrix multiplication
-

Read-Write

- Shared data updated by multiple tasks
- Requires synchronization

Example: Heuristic search in 15-Puzzle

- Shared **priority queue**
- Tasks:
 - insert states
 - remove best candidate

Implication

- Locks / atomic operations needed
 - Higher overhead
-

4. One-Way vs Two-Way Interactions

One-Way

- Only requester initiates interaction
- No interruption of source

Example

- Reading shared data
-

Two-Way

- Sender and receiver explicitly synchronize
- Required in message passing

Important

- Message passing **cannot support one-way interactions directly**
- Must be converted using:

- explicit sends
- polling
- acknowledgements

3.4 Mapping Techniques for Load Balancing

Mapping aims to:

1. **Minimize communication overhead**
2. **Minimize process idle time**

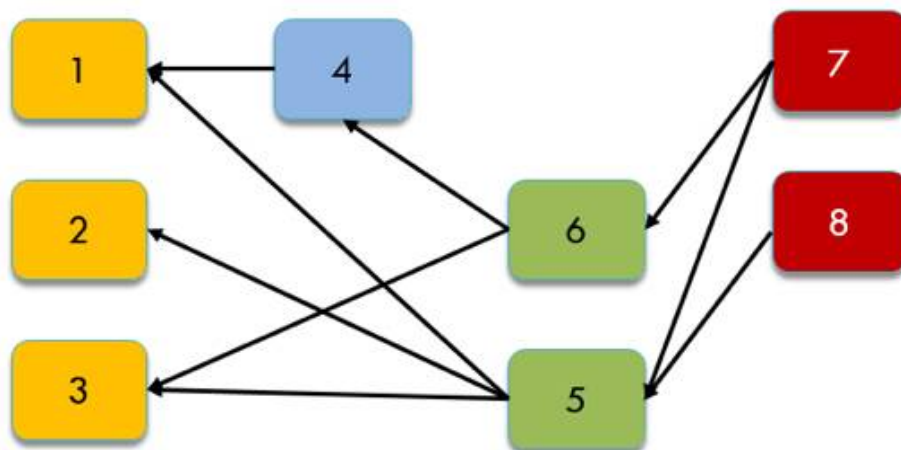
These goals **often conflict**, making mapping non-trivial.

Why Mapping is Hard

- Tasks may finish at different times
- Dependencies restrict concurrency
- Poor synchronization causes idle waiting



<https://www.researchgate.net/publication/351412734/figure/fig1/AS%3A1020333964398592%401620277982145/Example-of-mapping-of-DAG-tasks-onto-VMs-with-idle-time-slots-due-to-data-transfer.png>



Static vs Dynamic Mapping

Aspect	Static Mapping	Dynamic Mapping
Task knowledge	Known beforehand	Unknown / changing
Complexity	Easier	Harder
Overhead	Low	Higher
Load balance	May be poor	Usually better

3.4.1 Static Mapping Techniques

Used when:

- Task sizes known
- Interaction predictable

A. Mappings Based on Data Partitioning

Block Distributions



<https://www.researchgate.net/publication/3300516/figure/fig1/AS%3A339757914705922%401458016007822/Examples-of-regular-array-distributions-a-A-BLOCK-distribution-b-a-CYCLIC.png>

$$\begin{array}{c}
 \begin{array}{c} A \\ B \\ C \\ \{\omega\} \end{array}
 \begin{bmatrix}
 P_{AA} & P_{AB} & 0 & P_{A\{\omega\}} \\
 0 & P_{BB} & P_{BA} & P_{B\{\omega\}} \\
 0 & 0 & I & 0 \\
 0 & 0 & 0 & 1
 \end{bmatrix}
 \end{array}$$

- Contiguous blocks assigned to processes
- Excellent locality
- Poor for uneven workloads

1D and 2D Block Distribution



<https://www.researchgate.net/publication/317284273/figure/fig4/AS%3A576525846183940%401514465881237/Schematic-visualizations-of-a-two-dimensional-block-cyclic-distribution-used-in-the.png>

	Very dissatisfied	Somewhat dissatisfied	Neither dissatisfied nor satisfied	Somewhat satisfied	Very satisfied	Top 2 Box
Design	5.5%	14.3%	16.5%	36.4%	27.3%	50.0%
Hardware	4.7%	17.1%	17.3%	37.5%	23.4%	50.0%
Software	6.7%	16.8%	18.9%	38.7%	18.9%	37.5%
Price	11.3%	11.9%	19.3%	43.7%	13.8%	37.5%
Warranty	8.9%	19.85%	21.1%	44.6%	5.6%	37.5%
Average	7.4%	16.0%	18.6%	40.2%	17.8%	42.5%

Key Insight

- Higher dimensional partitioning:
 - increases concurrency
 - reduces communication

Block vs Block-Cyclic Distribution

LU Factorization Problem

Serial code (from description):

```

c
procedure COL_LU (A)
begin
  for k := 1 to n do
    for j := k to n do
      A[j, k] := A[j, k] / A[k, k];
    endfor;
    for j := k + 1 to n do
      for i := k + 1 to n do
        A[i, j] := A[i, j] - A[i, k] * A[k, j];
      endfor;
    endfor;
  endfor;
end COL_LU

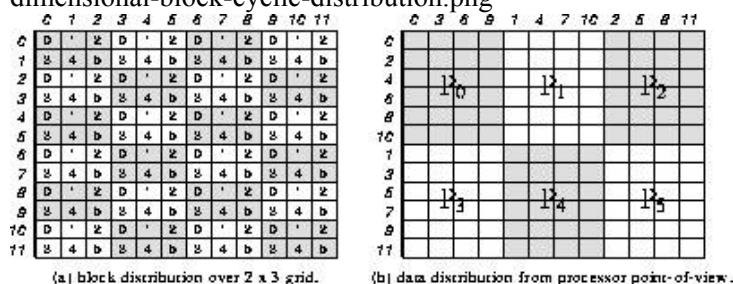
```

Issue

- Computation shifts to bottom-right
- Block distribution causes idle processes

Block-Cyclic Distribution

<https://www.researchgate.net/publication/2483870/figure/fig1/AS%3A279895600058374%401443743719474/Two-dimensional-block-cyclic-distribution.png>

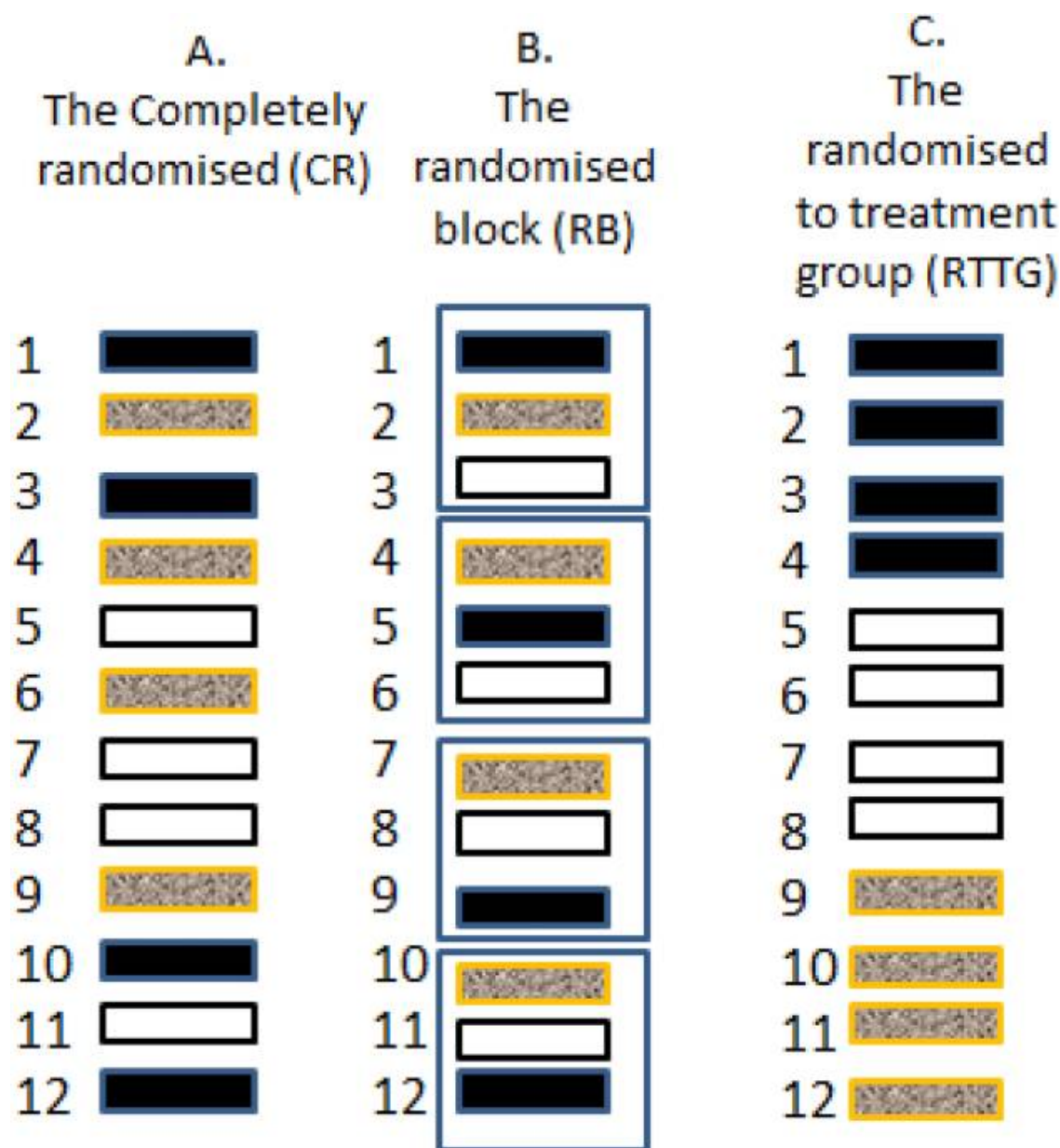


- Blocks distributed round-robin
- Each process gets work from entire matrix
- Reduces idling

Trade-off

- Too small blocks → poor locality
- Too large blocks → imbalance

Randomized Block Distribution



sparse matrix

1	3	0	0
0	4	6	0
2	0	7	8
0	5	0	9

COO

0	0	1	1	2	2	2	3	3
---	---	---	---	---	---	---	---	---

row_index

0	1	1	2	0	2	3	1	3
---	---	---	---	---	---	---	---	---

col_index

1	3	4	6	2	7	8	5	9
---	---	---	---	---	---	---	---	---

val

CSR

0	2	4	7	9
---	---	---	---	---

row_ptr

0	1	1	2	0	2	3	1	3
---	---	---	---	---	---	---	---	---

col_index

1	3	4	6	2	7	8	5	9
---	---	---	---	---	---	---	---	---

val

ELL

0	1	*
1	2	*
0	2	3
1	3	*

col_index

1	3	*
4	6	*
2	7	8
5	9	*

val

DIA

*	1	*
*	4	6
2	7	8
5	9	*

val

-2	0	1
----	---	---

offset

Used when:

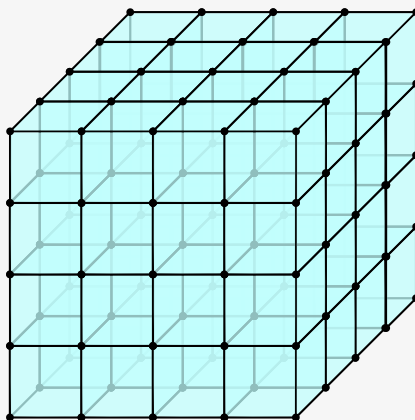
- Block-cyclic still causes imbalance
- Data sparsity patterns are skewed

B. Graph Partitioning

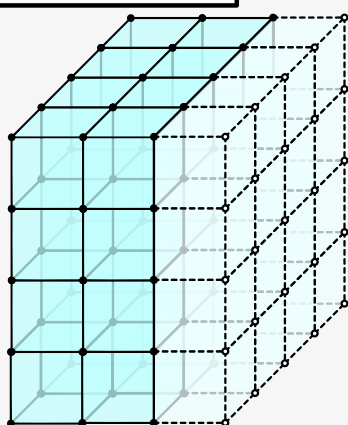


<https://www.researchgate.net/publication/382061860/figure/fig4/AS%3A11431281259119914%401720370647415/Parallel-computing-time-at-different-mesh-partitioning-schemes.ppm>

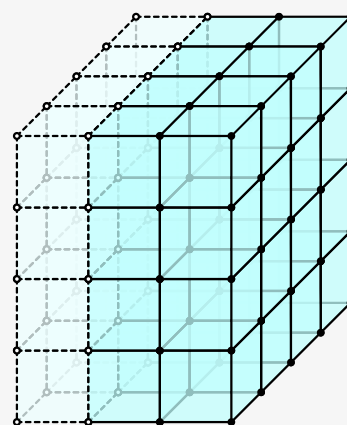
Original mesh



Partitioned mesh

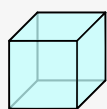


Domain 1



Domain 2

Legend



Owned cell



Ghost cell



Owned face



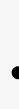
Ghost face



Owned edge



Ghost edge



Owned node



Ghost node

Used for:

- Sparse matrices
- Finite-element simulations
- Physical domain modeling

Goal

- Balance vertices
- Minimize edge cuts

Properties

- NP-complete problem
- Solved using heuristics

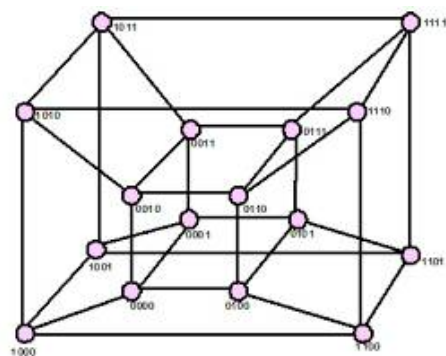
C. Mapping Based on Task Partitioning

Used when:

- Static task graph
- Known task sizes

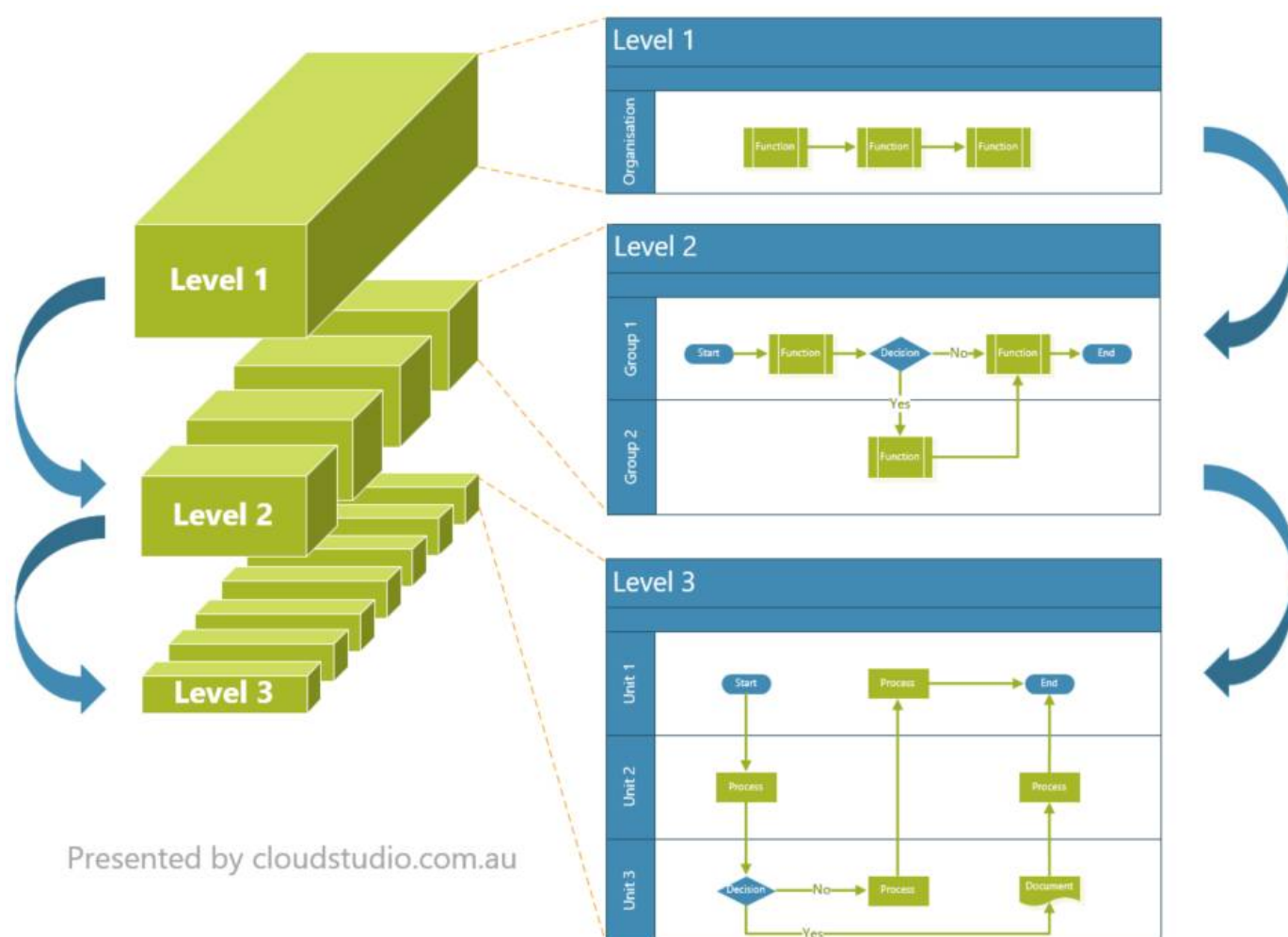
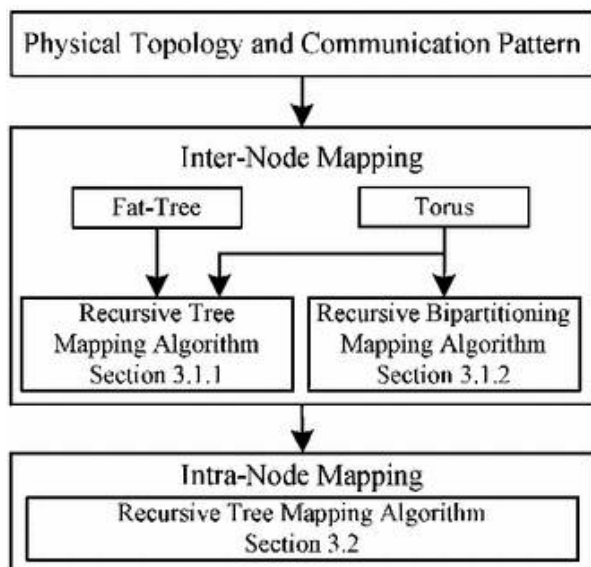
Binary Tree Task Graph

 https://www.researchgate.net/profile/Sergei-Shudler/publication/320678407/figure/fig1/AS%3A580526858014720%401515419796704/Task-dependency-graph-each-node-contains-the-task-time-and-the-highlighted-tasks-form_Q320.jpg



- Map dependent tasks to same process
- Reduce communication
- Preserve maximal concurrency

Hierarchical Mapping



Presented by cloudstudio.com.au

- Tasks decomposed into subtasks
- Different mapping at each level
- Common in:
 - Parallel quicksort
 - Sparse matrix factorization

3.4.2 Dynamic Mapping Techniques

Used when:

- Task sizes unknown
- Task graph dynamic
- High imbalance risk

Centralized Dynamic Mapping

Master-Worker Model

Example: Row Sorting

```
c
for (i = 1; i < n; i++)
    sort(A[i], n);
```

- Each iteration takes variable time
- Central pool of rows
- Idle workers fetch work

Techniques

- Self-scheduling
- Chunk scheduling
- Adaptive chunk sizing

Limitation

- Central bottleneck
- Poor scalability

Distributed Dynamic Mapping

- Processes exchange tasks among themselves
- No central authority

Key Design Questions

- Who initiates transfer?
- How much work to transfer?
- When to request work?

Advantages

- Scales better
- More complex

Suitability to Architectures

Architecture	Dynamic Mapping
Message passing	Needs coarse tasks
Shared memory	Supports finer tasks

NUMA Caveat

- Even shared memory has data-movement costs

Final Summary

- **Task characteristics** determine mapping feasibility
- **Interaction patterns** dictate communication overhead
- **Static mapping** is simple but brittle
- **Dynamic mapping** is powerful but complex
- **Block-cyclic & graph partitioning** are key load-balancing tools
- **Hierarchical mapping** enables multi-level parallelism

These sections form the **core decision framework** for designing efficient, scalable parallel algorithms across architectures.

If you want, I can next:

- convert this into **exam notes**
- create **comparison tables**
- or generate **PAGP-style viva questions**

Below is a **clear, structured, and exam-ready explanation** of **Sections 3.5 and 3.6** of *Principles of Parallel Algorithm Design*.

The content is **expanded conceptually, well formatted, and faithful to the original text**, with **examples and explanations added where reasoning is required**.

(No emojis used, as requested.)

3.5 Methods for Containing Interaction Overheads

In parallel programs, **interaction overhead** arises when concurrent processes:

- exchange data,
- synchronize,
- or contend for shared resources.

Even if a problem has abundant parallelism, **excessive interaction can nullify speedup**.

Therefore, designing a good parallel algorithm requires **actively controlling interaction overhead**, not just maximizing concurrency.

Interaction overhead depends mainly on:

1. **Volume of data exchanged**
2. **Frequency of interactions**
3. **Spatial and temporal interaction patterns**

This section discusses techniques that reduce interaction overhead by manipulating one or more of these factors.

3.5.1 Maximizing Data Locality

Core Idea

Perform as much computation as possible using **local data**, and **reuse data once fetched**.

This principle mirrors cache-locality optimizations in modern processors.

1. Minimizing Volume of Data Exchange

Reducing the *amount* of shared or nonlocal data accessed directly lowers interaction overhead.

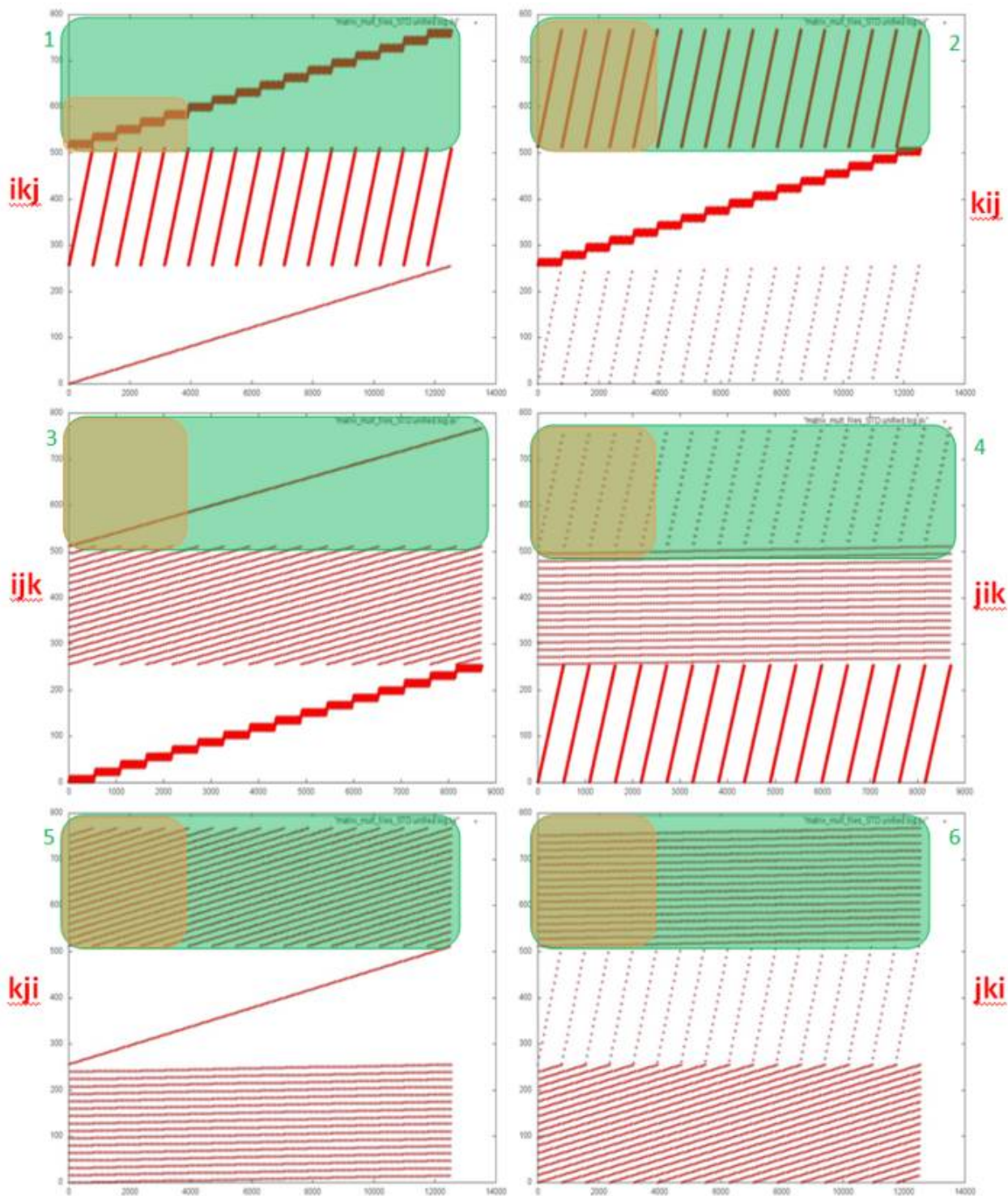
Example: Matrix Multiplication

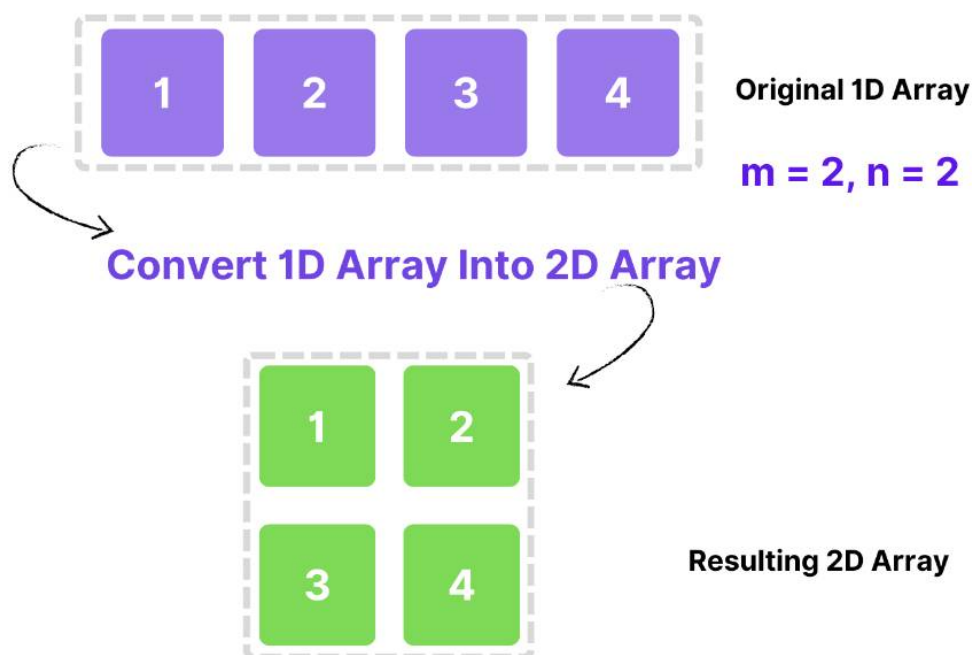
Recall from earlier discussion:

- **1-D partitioning**
Each process accesses:
 - its portion of matrix **A**
 - the **entire matrix B**
- **2-D partitioning**
Each process accesses:
 - only a **subset of A**
 - only a **subset of B**

Result:

- Shared data accessed per process is dramatically reduced.





Example: Parallel Dot Product Optimization

Naive approach:

- Each multiplication immediately updates a shared result
- Results in **n shared updates**

Optimized approach:

- Each process computes a **local partial sum**
- Final shared update occurs **once per process**

Effect

- Shared memory accesses reduced from **n** $\rightarrow$ **p**

This is a classic **local accumulation before global reduction** strategy.

2. Minimizing Frequency of Interactions

Even if total data volume remains the same, **frequent interactions are expensive** due to startup costs.

Strategy

- Access shared data in **larger chunks**
- Combine multiple small interactions into fewer large ones

Sparse Matrix-Vector Multiplication Optimization

Key observation:

- Matrix structure (nonzero pattern) is often **fixed**
- Only numerical values change between iterations

Optimized strategy

1. Scan matrix once to determine required vector entries
2. Collect all required nonlocal vector entries **in advance**
3. Perform multiplication **without further interaction**

This converts:

- many small, unpredictable interactions
into
- one predictable, bulk interaction

3.5.2 Minimizing Contention and Hot Spots

What is Contention?

Contention occurs when multiple processes attempt to access:

- the same memory block,
- the same communication link,
- or the same process

Only one access can proceed at a time → others must wait.

Example: Contention in 2-D Matrix Multiplication

When computing blocks $C_{i,j}$:

$$C_{i,j} = \sum_k A_{i,k} \times B_{k,j}$$

Problem:

- All processes computing the same **row of C** access the same block of **A**
- All processes computing the same **column of C** access the same block of **B**

Result:

- Severe contention

Contention-Free Reordering

By **changing the order** of block multiplication:

$$C_{i,j} = \sum_k A_{i,(j+k)\%p} \times B_{(j+k)\%p,j}$$

Effect:

- Each process accesses **different blocks**
- Eliminates simultaneous access to same data
- No contention

Key Insight

Contention can often be removed **without changing computation**, simply by **reordering accesses**.

Contention in Dynamic Mapping

Centralized load balancing (master-worker):

- Many workers request work from a single master
- Master becomes a bottleneck

Solution

- Use **distributed dynamic mapping**
 - Trade simplicity for scalability
-

3.5.3 Overlapping Computation with Interaction

Goal

Do useful work while waiting for data or messages.

Waiting time = wasted time.

Techniques for Overlap

1. Early Initiation of Interaction

- Start data transfer **before it is needed**
- Perform independent computation while data is in transit

Requires:

- predictable interaction pattern
 - static dependencies
-

2. Task Switching During Blocking

If one task is waiting:

- execute another ready task on the same process

This requires:

- finer granularity
 - careful balance (too fine → overhead)
-

3. Dynamic Load Balancing Anticipation

Instead of requesting work **after running out**:

- predict low workload
- request work **in advance**

Reduces idle time.

Hardware and Paradigm Support

Message Passing

- Non-blocking primitives (e.g., `MPI_Isend`, `MPI_Irecv`)
- Communication progresses while computation continues

Shared Memory

- Hardware prefetching
- Compiler-inserted prefetch instructions

Limitation

- Effectiveness depends on predictability of access patterns
-

3.5.4 Replicating Data or Computations

Data Replication

When:

- data is read-only

- accessed frequently
- irregular access pattern

Strategy

- Replicate data locally on each process
- Avoid repeated communication

Trade-Off

Benefit	Cost
Eliminates interaction	Increased memory usage
Simplifies code	Memory grows with process count

Best suited for:

- small read-only data
- message-passing systems

Replicated Computation

Sometimes:

- recomputing data is cheaper than communicating it

Example: FFT Twiddle Factors

- Each process computes the twiddle factors it needs locally
- Avoids sharing and synchronization

Even if more arithmetic is done:

- overall execution may be faster

3.5.5 Using Optimized Collective Interaction Operations

Many parallel algorithms repeatedly use **common communication patterns**, such as:

- broadcast
- reduction
- gather
- scatter
- barrier synchronization

These are called **collective operations**.

Why Use Collectives?

- Highly optimized
- Minimize contention
- Exploit hardware topology
- Provided by libraries (e.g., MPI)

Examples

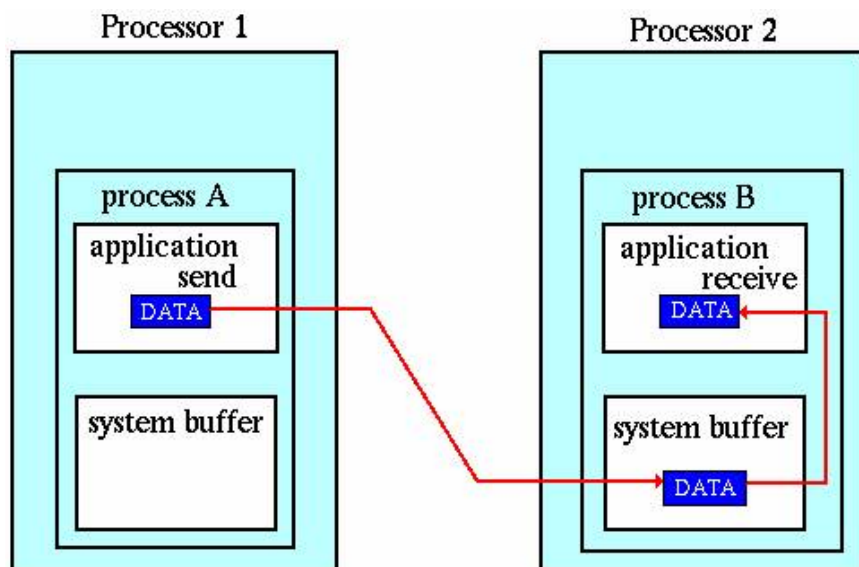
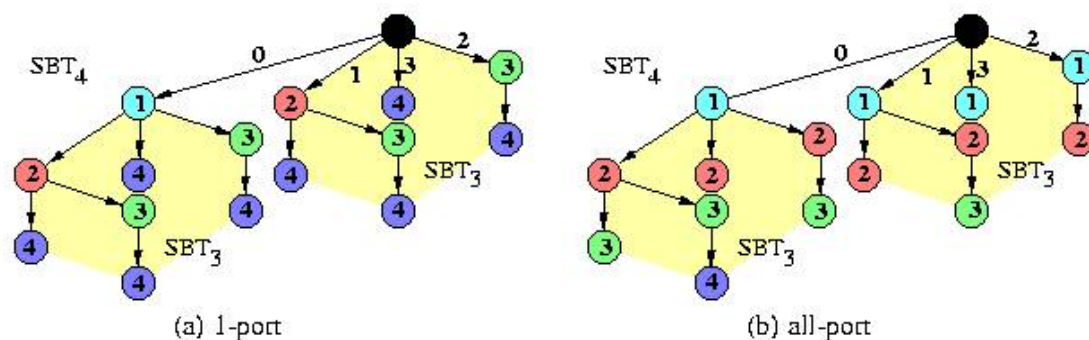
- `MPI_Bcast`
- `MPI_Reduce`
- `MPI_Barrier`

Programmer focuses on **what**, not **how**.

3.5.6 Overlapping Interactions with Other Interactions

Sometimes, **communication** can overlap with **communication**.

Broadcast Example (4 Processes)



Path of a message buffered at the receiving process

Efficient Method

- $P0 \rightarrow P2$
- Then:
 - $P0 \rightarrow P1$
 - $P2 \rightarrow P3$
- Completes in **2 steps**

Naive Method

- $P0 \rightarrow P1 \rightarrow P2 \rightarrow P3$
- Takes **3 steps**

Pipeline Broadcast for Multiple Messages

If multiple broadcasts occur:

- naive method allows pipelining
- may outperform optimized method

Key Insight

Sometimes, custom communication beats library collectives when interaction patterns are known.

3.6 Parallel Algorithm Models

A **parallel algorithm model** is a reusable way of structuring:

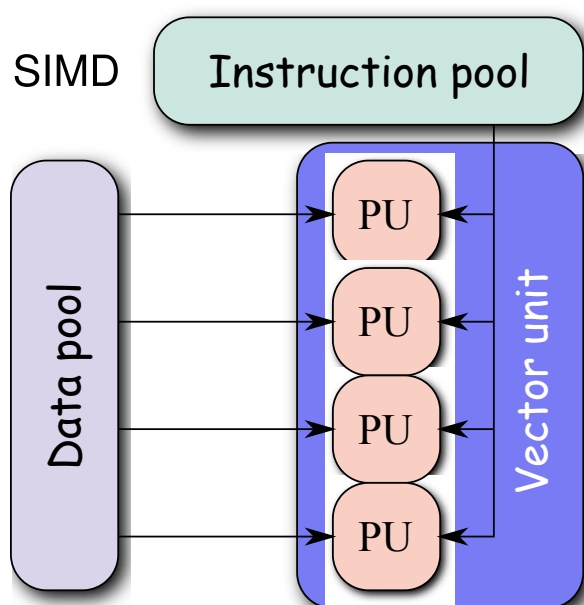
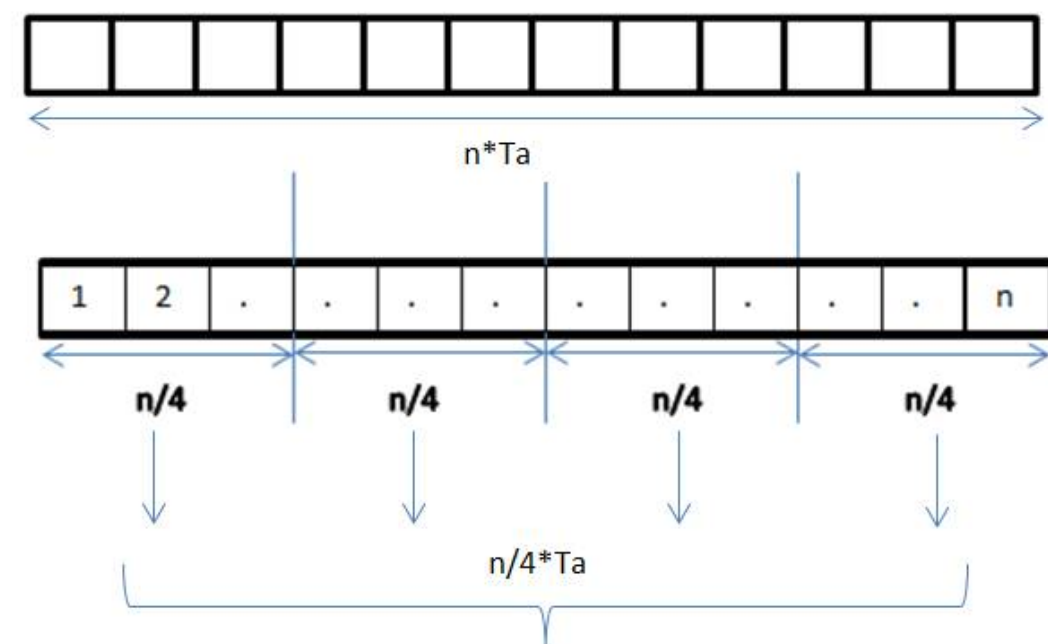
- decomposition
- mapping
- interaction control

Each model suits a particular class of problems.

3.6.1 Data-Parallel Model

Characteristics

- Same operation applied to different data
- Tasks perform identical computations
- Mapping usually static



Advantages

- Simple
- Easy load balance
- Scales with problem size

Example

- Dense matrix multiplication
- Image processing
- Vector operations

3.6.2 Task Graph Model

Characteristics

- Explicit task-dependency graph
- Tasks may be heterogeneous
- Mapping considers interaction patterns

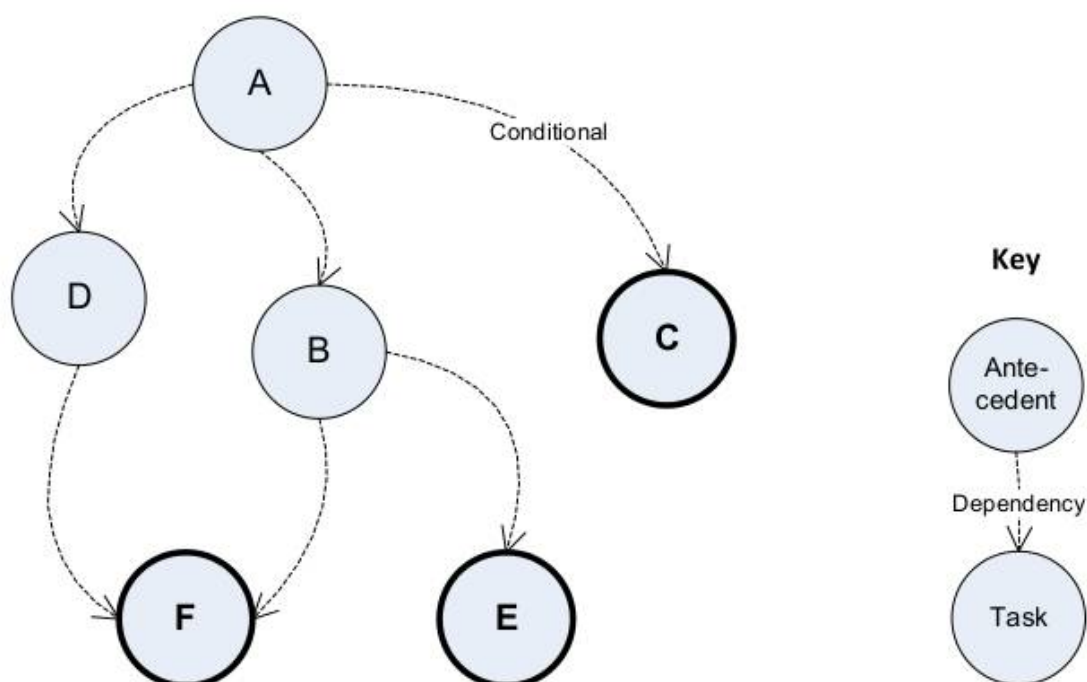
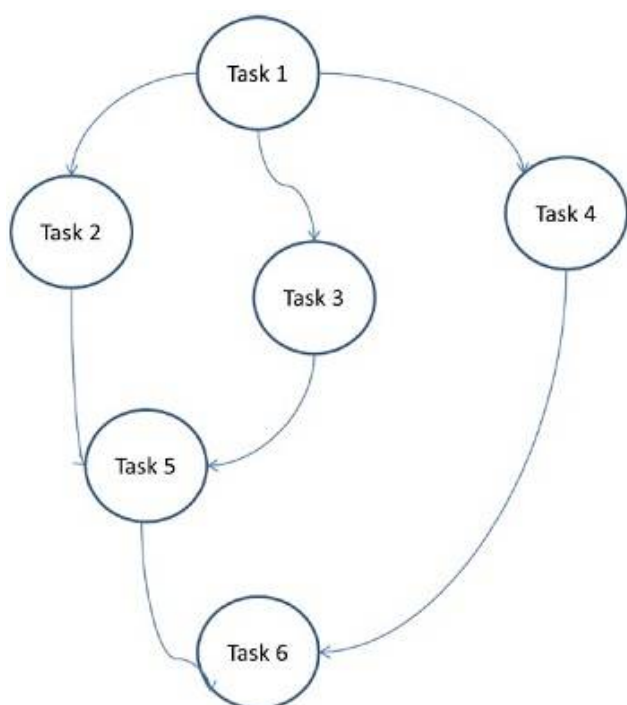


Figure 1: A task graph with dependencies and a conditional dependency.



Used When

- Data movement cost is high

- Task dependencies dominate structure

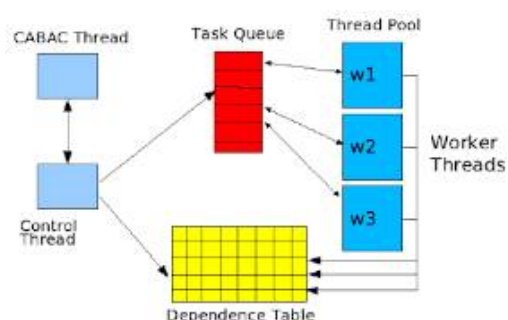
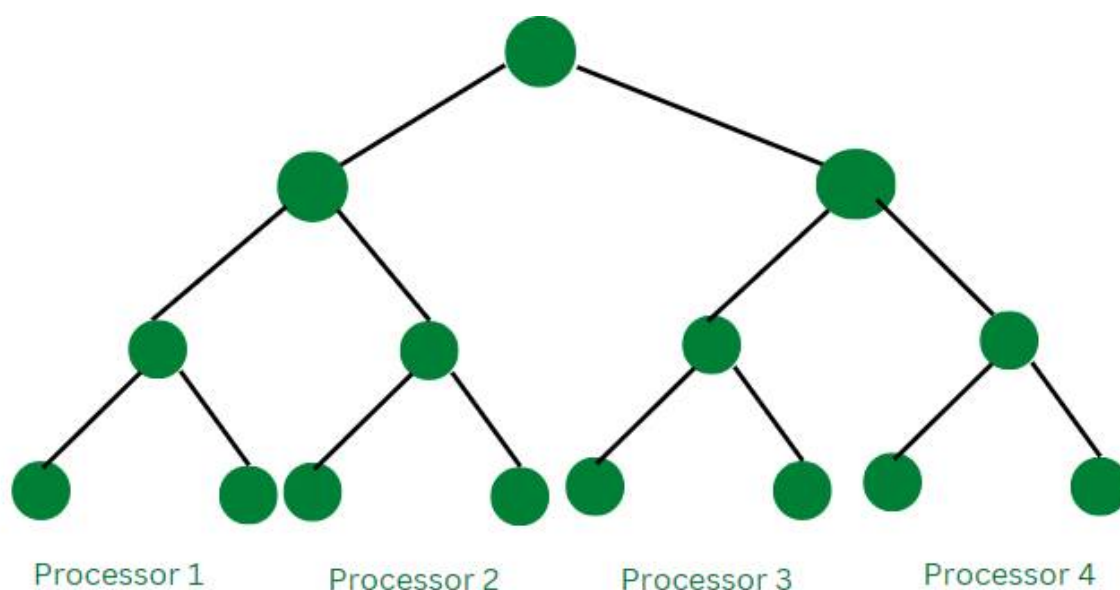
Examples

- Parallel quicksort
- Sparse matrix factorization
- Divide-and-conquer algorithms

3.6.3 Work Pool (Task Pool) Model

Characteristics

- No fixed mapping
- Tasks dynamically assigned
- Centralized or distributed pool



Suitable When

- Task sizes unpredictable
- High load imbalance risk

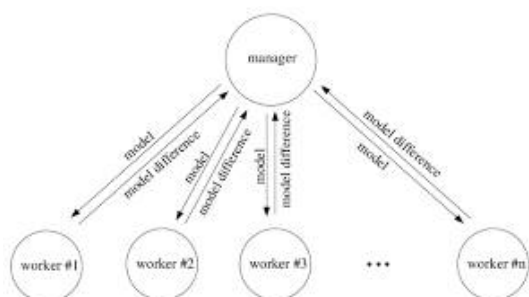
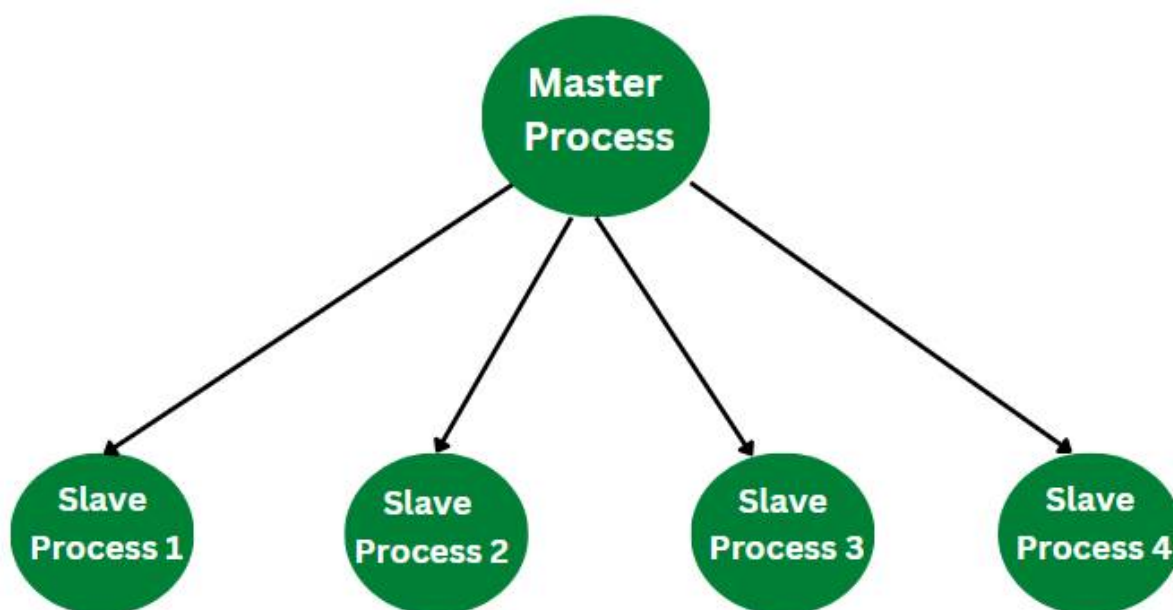
Examples

- Parallel loop scheduling
- Parallel search
- Tree exploration

3.6.4 Master-Slave (Manager-Worker) Model

Characteristics

- Master generates or assigns work
- Workers execute tasks
- Interaction is naturally two-way



Risks

- Master bottleneck
- Requires coarse tasks

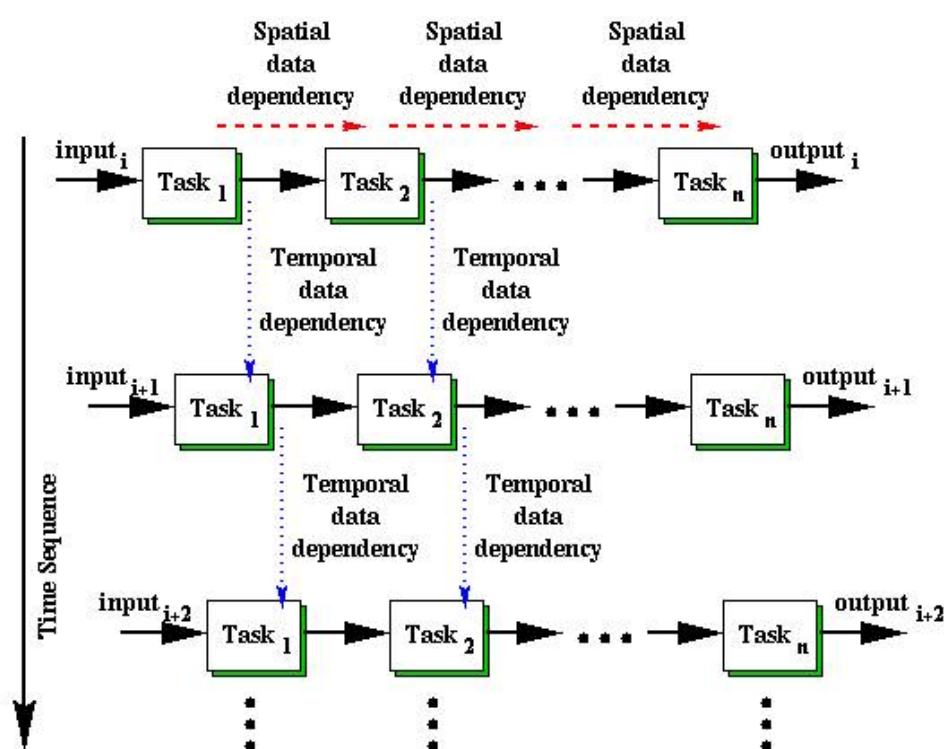
Enhancements

- Hierarchical masters
- Asynchronous communication

3.6.5 Pipeline / Producer-Consumer Model

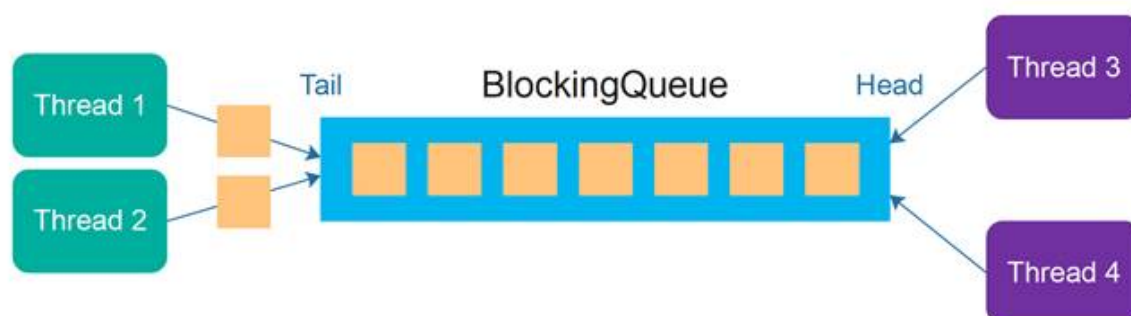
Characteristics

- Stream of data flows through stages
- Different stages work concurrently
- Emphasizes stream parallelism



Producer Threads

Consumer Threads



Trade-Off

- Large tasks → slow pipeline fill

- Small tasks → high interaction overhead

Example

- LU factorization
 - Multimedia processing
 - Instruction pipelines
-

3.6.6 Hybrid Models

Most real algorithms are **hybrids**.

Examples:

- Task graph + data parallelism
- Pipeline + data parallel stages
- Recursive tasks + work pool

Example

- Parallel quicksort:
 - recursive task graph
 - data-parallel partitioning
 - dynamic load balancing
-

Final Takeaway

Sections **3.5** and **3.6** complete the **full design loop** of parallel algorithms:

1. Expose concurrency
2. Map tasks efficiently
3. Control interaction overhead
4. Choose the right algorithm model

A good parallel algorithm is not defined by **how many processors it uses**, but by **how intelligently it minimizes unnecessary interaction while exploiting concurrency**.

If you want next, I can:

- condense this into **exam revision notes**
- create **short answer + long answer questions**
- or map **each model to PAGP lab-style examples**